KU LEUVEN

FACULTEIT INDUSTRIËLE
INGENIEURSWETENSCHAPPEN

TECHNOLOGIECAMPUS GENT

Project: Boids

Labo Geavanceerde Computerarchitectuur

Kjell NAESSENS
Beau FORREZ

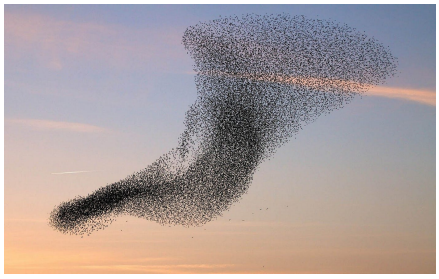
Labo verantwoordelijke: T. Feys

Academiejaar 2025-2026

The source code is available on GitHub.[1]

1 Context

Het gedrag van groepen dieren, zoals vogels, vertoont vaak complexe patronen uit eenvoudige regels. Een bekend model om dit gedrag te bestuderen is het zogenaamde Boids-model, geïntroduceerd door Reynolds in 1986 [2]. Het Boids-algoritme simuleert het collectieve gedrag van een groep vogels door elke boid (bird-oid object) te laten reageren op de posities en snelheden van zijn directe buren.



Figuur 1: Boids in de natuur. [1]

Het model bestaat uit drie kernregels:

1. **Separation:** boids bewegen weg van andere boids die te dichtbij komen, zodat botsingen worden vermeden.
2. **Alignment:** boids passen hun snelheid en richting aan op basis van de gemiddelde snelheid van naburige boids.
3. **Cohesion:** boids bewegen naar het middelpunt van hun naburige boids om de groep bij elkaar te houden.

Door deze eenvoudige regels toe te passen, ontstaan er spontaan complexe en natuurlijke flocking-patronen zonder dat een centrale coördinator aanwezig is. Dit fenomeen illustreert hoe lokale interacties kunnen leiden tot georganiseerde globale structuren.

Voor ons project vormt het Boids-algoritme volgens V. Hunter Adams [3] de basis voor het implementeren van een simulatie. Door het model te implementeren op CPU en GPU, kunnen we onderzoeken hoe schaalbaarheid, performance en real-time visualisatie elkaar beïnvloeden.

2 CPU versie

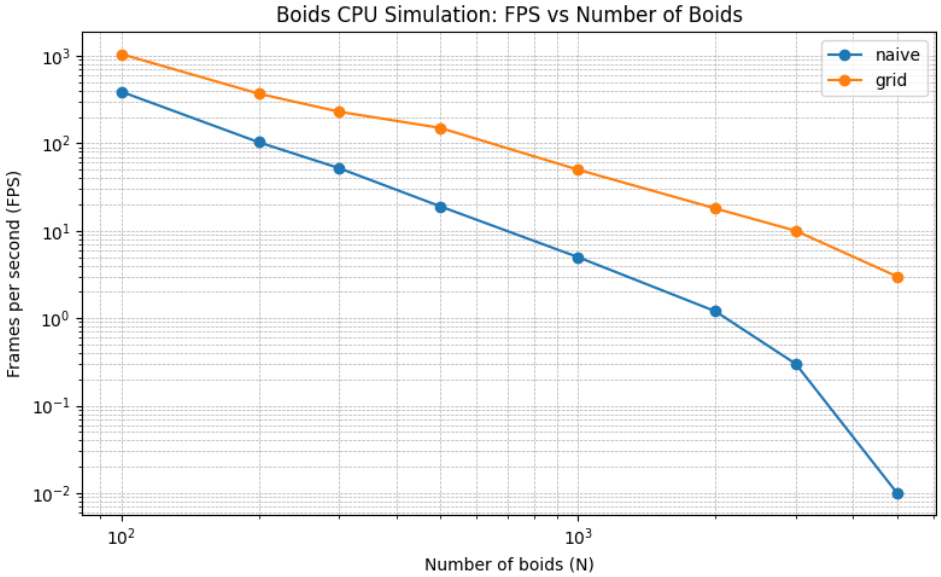
De eerste implementatie van het Boids-model werd uitgevoerd op de CPU, in Python met behulp van de packages NumPy en matplotlib (gebaseerd op [3]). Deze versie vormt het referentiepunt voor verdere optimalisaties en wordt gebruikt om de beperkingen van een zuiver sequentiële uitvoering te onderzoeken. De volledige simulatie draait in één thread, aangezien de Global Interpreter Lock van Python verhindert dat meerdere Python-threads daadwerkelijk parallel kunnen rekenen op verschillende CPU-kernen. Dit maakt de CPU-implementatie bijzonder geschikt als baseline: er wordt getoond hoe de rekencomplexiteit van het Boids-algoritme schaalt wanneer het aantal boids N toeneemt.

[1] `https://github.com/beauforr/Boids-Simulation`

De implementatie bestaat uit twee varianten voor het bepalen van de buren van elk boid:

1. **Naïeve methode**: elke boid vergelijkt zijn positie met alle andere boids. Dit resulteert in een rekencomplexiteit van $O(N^2)$ waardoor de performantie snel daalt bij grote aantallen boids. Dit is de 'volledige' methode.

2. **Grid-gebaseerde methode**: de wereld wordt onderverdeeld in cellen die ongeveer overeenkomen met de visuele straal van een boid. Elke boid wordt in een cel geplaatst, en enkel boids uit de omliggende negen cellen worden bekeken. Hierdoor daalt de effectieve complexiteit naar ongeveer $O(N)$, waardoor de simulatie bij hogere waarden van N aanzienlijk beter schaalt. Dit is een vorm van spatial hashing, een benadering die voor goede optimalisatie zorgt.



Figuur 2: Vergelijking van de FPS voor naïeve en grid-methode op CPU

In beide methoden worden de drie kernregels van het Boids-model (separation, alignment, cohesion) berekend op basis van naburige boids. De simulatiestatus (posities, snelheden en biasgroepen) wordt opgeslagen in NumPy-arrays om efficiënt te kunnen rekenen. Daarnaast wordt rekening gehouden met randvoorwaarden, snelheidslimieten en eventuele biasrichtingen.

De visualisatie verloopt via matplotlib.animation, waarbij elke frame opnieuw wordt getekend. Tijdens de uitvoering wordt de rekentijd per frame gemeten en via een gemiddelde over dertig frames omgezet naar stabiele FPS-waarden. Deze benchmarks (figuur 2) tonen duidelijk het verschil tussen de naïeve $O(N^2)$ aanpak en de efficiëntere grid-methode, die quasi lineair schaalt en daardoor real-time blijft bij grotere aantallen boids.

Hoewel de CPU-versie volledig functioneel is en een goede baseline vormt, worden de beperkingen van sequentiële verwerking snel zichtbaar, vooral bij hogere resoluties en veel boids. Hierdoor is de overstap naar een parallelle uitvoering op de GPU dus logisch.

3 GPU versie

Om de beperkte schaalbaarheid van de CPU-implementatie te verbeteren, werd het Boids-algoritme geëxporteerd naar de GPU met behulp van PyCUDA. Het kernidee is dat elke boid volledig onafhankelijk kan worden bijgewerkt, waardoor de berekeningen uitstekend paralleliseerbaar zijn.

In plaats van voor elke boid sequentieel de buren te doorzoeken, wordt op de GPU een thread gelanceerd voor een aantal boids. Deze thread voert exact dezelfde drie regels uit: separation, alignment en cohesion, maar nu in parallel. Voor N boids worden dus N CUDA-threads geactiveerd, wat de effectieve rekentijd drastisch verlaagt.

De implementatie maakt gebruik van:

- **Device arrays** voor posities, snelheden en biaswaarden
- **Een CUDA-kernel** die de volledige update uitvoert
- **Global memory access** voor het uitlezen van alle boids (naïeve $O(N^2)$ versie
- **Thread blocks van 128 of 256 threads**, afgestemd op typische CUDA-architecturen. Bij 128 en 256 threads per block werd de hoogste FPS verkregen, dus daarom werden deze waarden gekozen.

Hoewel de buurdetectie nog steeds $O(N^2)$ is, wordt deze complexiteit nu verdeeld over honderden tot duizenden CUDA-cores, waardoor de effectieve uitvoering per frame sterk versnelt.

De essentie van de GPU-implementatie zit niet in de Python-aanroep, maar in de CUDA-kernel zelf. Listing 1 toont het kernfragment van deze kernel en illustreert expliciet hoe parallellisatie, geheugenstructuur en complexiteit samenkomen.

Listing 1: Kern van de CUDA-kernel voor het updaten van boids

```
__global__ void update_boids(float *pos_x, float *pos_y,
                             float *vel_x, float *vel_y,
                             int N, float visual_range, float dt)
{
    // 1. Unieke index: 1 thread = 1 boid
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= N) return;

    // 2. Laad data van deze boid in registers
    float xi = pos_x[i];
    float yi = pos_y[i];
    float vxi = vel_x[i];
    float vyi = vel_y[i];

    float ax = 0.0f;
```

```
    float ay = 0.0f;

    // 3. Naieve O(N^2) buurloop
    for (int j = 0; j < N; ++j) {
        if (j == i) continue;

        float dx = xi - pos_x[j];
        float dy = yi - pos_y[j];

        if (fabsf(dx) < visual_range &&
            fabsf(dy) < visual_range) {
            // separation / alignment / cohesion
            ax += dx;
            ay += dy;
        }
    }

    // 4. Schrijf resultaat terug naar global memory
    vel_x[i] = vxi + ax * dt;
    vel_y[i] = vyi + ay * dt;

    pos_x[i] = xi + vel_x[i] * dt;
    pos_y[i] = yi + vel_y[i] * dt;
}
```

Deze kernel maakt expliciet duidelijk dat de berekening per boid onafhankelijk is en dat de totale rekenlast voortkomt uit de geneste structuur van threads en de interne buurloop. Deze GPU-versie genereert uiteindelijk dezelfde bewegingen als de CPU-versie, maar met een sterk verhoogde framerate en de mogelijkheid om veel grotere aantallen boids te simuleren. De visualisatie gebeurt nog via matplotlib, waarbij na elke frame enkel de posities worden teruggehaald van de GPU.

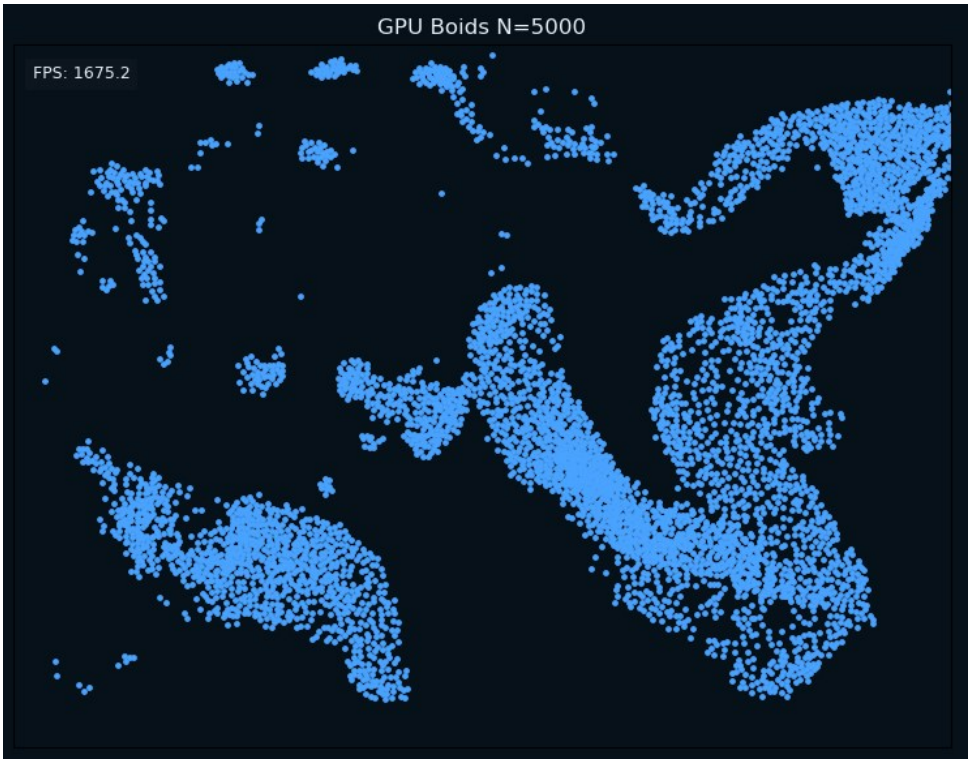
De CUDA code werd geoptimaliseerd zoals in de labo's:

- **Planaire arrays** In plaats van de boids als een lijst van structuren op te slaan, worden posities en snelheden doorgegeven als afzonderlijke arrays (`pos_x`, `pos_y`, `vel_x`, `vel_y`). Hierdoor liggen de data-elementen die door opeenvolgende threads worden aangesproken lineair in het geheugen.
- **Coalesced memory access:** Dankzij deze planaire representatie kunnen threads binnen één warp (32 threads) hun gegevens ophalen via één of enkele gecoördineerde geheugentransacties, in plaats van 32 afzonderlijke toegangspogingen. Dit benut de beschikbare bandbreedte van het global memory aanzienlijk efficiënter.
- **Volledige warps:** De gekozen blockgrootte van 128 of 256 threads is een veelvoud van

de warp-grootte (32). Hierdoor worden voornamelijk volledige warps uitgevoerd, wat leidt tot een hoge occupancy van de streaming multiprocessors en minimale verspilling door inactieve threads, behalve eventueel in het laatste block.

Kritische reflectie: De huidige implementatie voert alle berekeningen uit in het global memory. Hoewel dit geheugen relatief traag is, blijft de code hierdoor eenvoudig en overzichtelijk. In een vervolgstap werd een optimalisatie met **shared memory** en tiling geïmplementeerd, waarbij subsets van boids per thread block tijdelijk in het on-chip geheugen werden geladen om het aantal globale geheugenaccesses te reduceren.

In de praktijk leverde deze aanpak echter geen meetbare prestatieverbetering op. Waarschijnlijk woog de snellere geheugentoegang niet op tegen de bijkomende overhead. We voerden ook een test uit met striding, maar ook hier was geen meetbare verbetering te verkrijgen.



Figuur 3: Visualisatie van de boids-simulatie op GPU

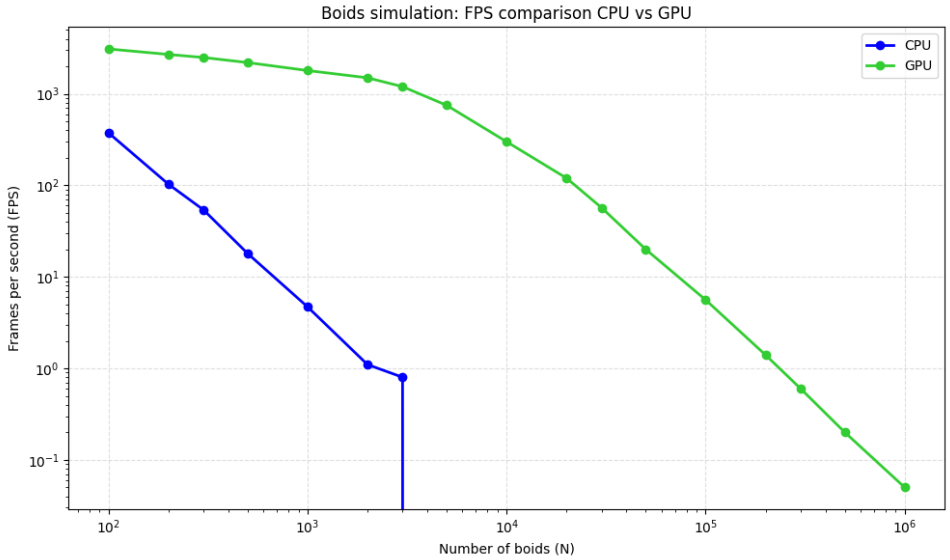
4 Vergelijking CPU en GPU

Om de prestaties van beide implementaties te evalueren, werden benchmarks uitgevoerd op verschillende aantallen boids. Voor iedere waarde van N werd de gemiddelde FPS gemeten over een groot aantal frames, volgens de labo-aanpak voor performantiemetingen.

De CPU-versie gebruikt een sequentiële $O(N^2)$ buurdetectie, terwijl de GPU-versie dezelfde berekening uitvoert maar parallel op honderden CUDA-cores. Hierdoor stijgt de performantie aanzienlijk bij grotere aantallen boids.

4.1 Benchmarkresultaten

De gemeten FPS-waarden voor beide implementaties zijn weergegeven in Figuur 4. Deze resultaten tonen duidelijk aan dat de CPU-versie snel terugvalt bij grotere aantallen boids, terwijl de GPU-versie dankzij massale parallelisatie quasi lineair schaalgedrag vertoont.



Figuur 4: Vergelijking van de FPS voor CPU- en GPU-implementaties van het Boids-model

4.2 Conclusie

De belangrijkste bevindingen op basis van de uitgevoerde metingen zijn:

- De CPU-implementatie schaalt zeer slecht en vertoont een snelle prestatie-instorting door de $O(N^2)$ complexiteit van de boids-interacties.
- Vanaf enkele honderden boids daalt de CPU-FPS sterk, en vanaf $N \geq 3000$ wordt geen bruikbare framerate meer gehaald.

- De GPU-implementatie behoudt daarentegen hoge en stabiele FPS-waarden voor kleine en middelgrote aantallen boids, met meer dan 1000 FPS tot ongeveer $N = 3000$.
- Ook bij zeer grote aantallen blijft de GPU functioneel: bij $N = 50\,000$ wordt nog steeds ongeveer 20 FPS gehaald, terwijl de CPU al lang volledig faalt.
- Zelfs bij extreme schaal ($N = 1\,000\,000$) blijft de GPU-simulatie uitvoerbaar, zij het met een sterk verlaagde framerate.

Deze resultaten bevestigen dat massale parallelisatie essentieel is voor boids-simulaties. Terwijl de sequentiële CPU-benadering snel zijn limiet bereikt, maakt de GPU het mogelijk om de simulatie over meerdere ordes van grootte groter te schalen.

5 Uitbreiding: 3D Foids met predator

Om het Boids-model verder uit te breiden naar een complexer en realistischer systeem, werd een driedimensionale variant ontwikkeld: **Foids** (fish-oids). In tegenstelling tot vogels bewegen de vissen in volledig 3D binnen een begrensd volume (aquarium), waardoor de simulatie een grotere mate van ruimtelijke vrijheid vereist. Bovendien werd een **predator**-entiteit toegevoegd, die het gedrag van de zwerm op een natuurlijke manier verstoort en zo de emergentie van vluchtgedrag zichtbaar maakt.

De implementatie is gebaseerd op dezelfde Boids-regels (separation, alignment, cohesion), maar werd aangepast voor een 3D-omgeving en uitgebreid met twee extra elementen:

1. **Volledig 3D-ruimtegebruik**: elke foid heeft een positie- en snelheidsvector in drie dimensies. De wereld is een gesloten doos met afmetingen $W \times H \times D$, waarbij de randen via een turn-factor worden afgedwongen.
2. **Predator-ontwijking**: vissen detecteren de predator binnen een ruim bereik en wijken sterk uit, waarbij de normale flockingregels ondergeschikt worden aan de nood om te vluchten.

5.1 3D Flockingmodellering

Voor het genereren van realistische 3D-beweging wordt elke foid geïnitieerd met een willekeurige positie en richting, gelijkmatig verdeeld over het volume. De snelheden worden begrensd tussen een minimum- en maximumwaarde om instabiliteit te voorkomen. De biasgroepen van de 2D-boids werden behouden, maar worden nu onderverdeeld per kleur in biasgroepen. We kozen ervoor om deze biasgroepen een voorkeur te geven om met hetzelfde type te groeperen, en groepen van een ander kleur te vermijden.

Net als in de 2D-GPU-versie wordt de update van elke foid volledig op de GPU uitgevoerd via een CUDA-kernel. Deze kernel verwerkt de standaardinteracties:

- **Separation**: vermijden van nabije foids; vijandige groepen genereren een sterkere afstotingskracht.
- **Alignment**: uitlijnen met de lokale buurtnetwerksnelheid.
- **Cohesion**: bewegen naar het lokale zwaartepunt van de groep.

Om computationele stabiliteit te behouden wordt ook in 3D de snelheid genormaliseerd en begrensd na elke update. Tenslotte worden Foids binnen de grenzen van de simulatiedoos gehouden via een turn-factor die afremt of bijstuurt in de richting van het midden wanneer de rand wordt benaderd.

5.2 Predator-interactie

Een belangrijke toevoeging is de **predator**, een enkel object dat zich sneller voortbeweegt dan de foids en continu door de 3D-omgeving beweegt. De predator heeft twee functies:

1. **Vroegtijdige detectie**: foids detecteren de predator binnen een grote straal r_{pred}, veel groter dan hun normale visuele bereik.
2. **Vluchtgedrag**: een sterke afstotingskracht wordt toegepast, die de standaard flockingregels tijdelijk overheerst, zodat de groep in paniek van de predator weg zwemt.

De vector van de foid naar de predator wordt genormaliseerd en vervolgens vermenigvuldigd met een *predator_avoid_factor*. Door de kracht te normaliseren ontstaat een consistente en directionele vluchtbeweging, ongeacht de afstand binnen de detectiestraal. Hierdoor ontstaan dynamieken zoals schrikgolven, explosieve uitwaaieringen en groepsverdichting, vergelijkbaar met echt visgedrag.

5.3 CUDA-kernel

De volledige update, inclusief alle boidregels en predatorlogica, wordt uitgevoerd op de GPU in de kernel `update_boids_3d`. De kernel verwerkt per thread één foid, waardoor de update 3D-parallel verloopt over duizenden vissen:

Listing 2: Kernlogica van de 3D CUDA-kernel voor boids met predator

```
__global__ void update_boids_3d(...)
{
    // 1. Unieke index per thread (1 boid per thread)
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= N) return;

    // Haal eigen data naar registers
    float xi = pos_x[i];
```

```
float yi = pos_y[i];
float zi = pos_z[i];

float vxi = vel_x[i];
float vyi = vel_y[i];
float vzi = vel_z[i];

// 2. Loop over alle buren (na ef O(N^2))
for (int j = 0; j < N; ++j) {
    if (j == i) continue;

    float dx = xi - pos_x[j];
    float dy = yi - pos_y[j];
    float dz = zi - pos_z[j];
    float dist2 = dx * dx + dy * dy + dz * dz;

    // Check Field of View (FOV)
    if (dist2 < visual_range * visual_range) {
        // Dot product voor kijkhoekberekening
        float dot = (-dx * vxi) + (-dy * vyi) + (-dz * vzi);

        // ... logica voor separation / alignment / cohesion ...
    }
}

// 3. Predator avoidance (vluchtgedrag)
float pdx = xi - pred_x;
float pdy = yi - pred_y;
float pdz = zi - pred_z;
float pdist2 = pdx * pdx + pdy * pdy + pdz * pdz;

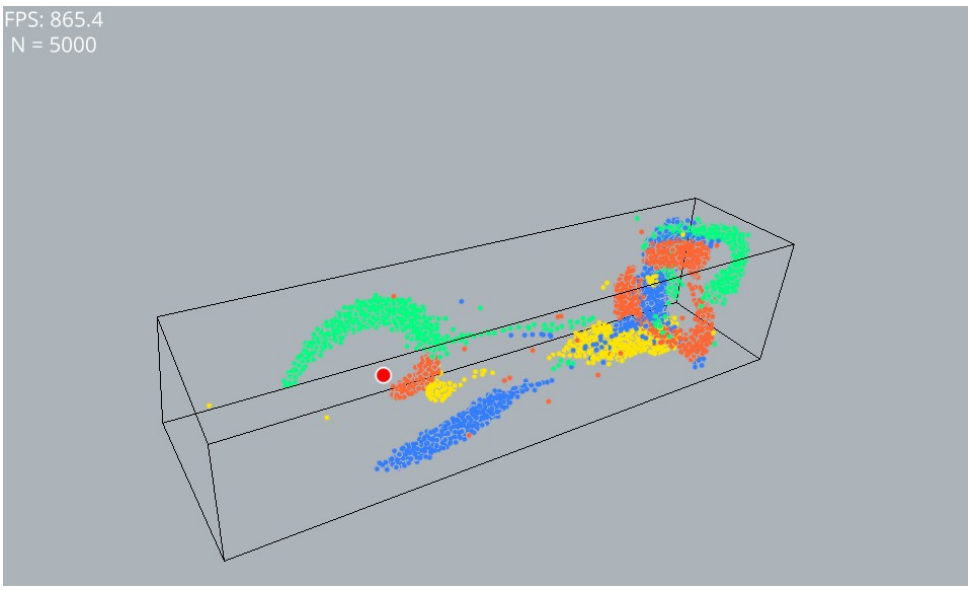
if (pdist2 < pred_range * pred_range) {
    // Sterke afstotende kracht weg van predator
    float inv_pdist = 1.0f / sqrtf(pdist2);
    new_vx += (pdx * inv_pdist) * pred_avoid_factor;
    new_vy += (pdy * inv_pdist) * pred_avoid_factor;
    new_vz += (pdz * inv_pdist) * pred_avoid_factor;
}

// 4. Schrijf nieuwe posities terug naar global memory
pos_x[i] = xi + new_vx * dt;
```

```
    pos_y[i] = yi + new_vy * dt;
    pos_z[i] = zi + new_vz * dt;
}
```

Binnen de kernel worden eerst de standaard flockingregels toegepast. Vervolgens wordt het predator-vluchtgedrag toegevoegd, waarbij de foid een sterk afstoting krijgt wanneer de predator binnen bereik is. Na deze stap worden de snelheidslimieten toegepast en wordt de positie geüpdatet.

5.4 Visualisatie in 3D



Figuur 5: Visualisatie van het 3D-foids model

De simulatie gebruikt `vispy` om Foids en predator in real-time weer te geven. Hierbij merkten we dat `matplotlib` te veel rekenwerk had waardoor het proces CPU-bound zou worden. Door gebruik te maken van `vispy` blijft de taak GPU-bound. De wereld wordt voorgesteld als een transparante doos, waarin:

- foids als kleine, gekleurde punten verschijnen;
- de predator als een groot, rood marker wordt weergegeven;
- de camerahoek vrij geroteerd kan worden tijdens de simulatie.

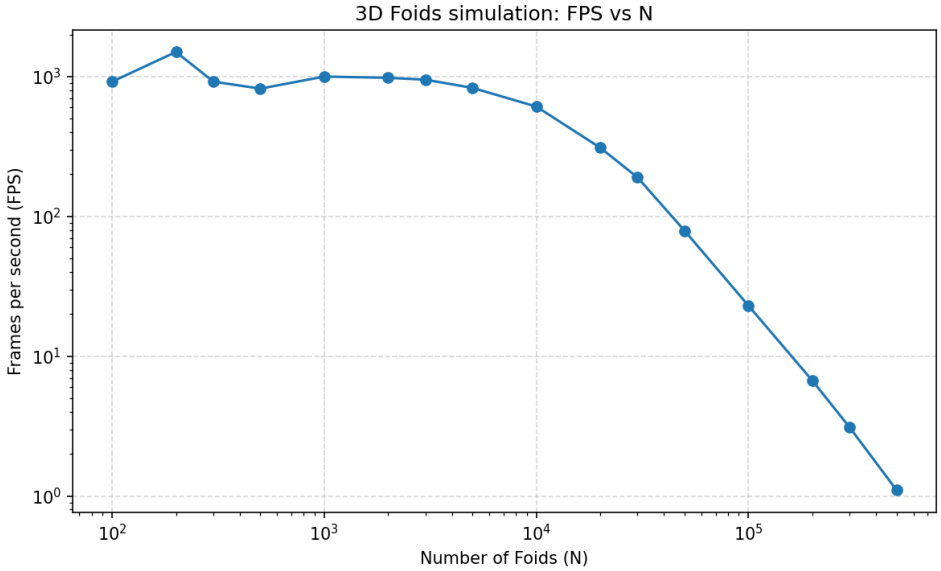
De GPU berekent continu nieuwe posities, die elk frame naar de CPU worden teruggestuurd voor visualisatie.

5.5 Resultaat

Het toevoegen van een predator maakt het gedrag veel dynamischer en realistischer. De groep beweegt niet langer als een uniforme massa maar vertoont complexe patronen zoals:

- groepsversnelling en samenklontering bij dreiging,
- ontwijkende spiralen en uitwaaierstructuren,
- spontane splitsingen van de school wanneer de predator door de groep beweegt.

Deze uitbreiding toont duidelijk de flexibiliteit van het Boids-model in een 3D-omgeving, en benadrukt hoe lokale interactieregels kunnen leiden tot indrukwekkend realistisch collectief gedrag.



Figuur 6: Performantie van het 3D-model

Voor de 3D simulatie hebben we geen CPU-referentie. We kunnen echter wel vaststellen dat de simulatie tot 10000 foids rond de 1000 FPS blijft, en vanaf dan sterk afneemt met toenemende foids-aantallen.

Referenties

[1] R. Zandie, “Simulating bird flock behavior in python using boids.” Online, May 2019. Published 28 May 2019. Available: https://medium.com/better-programming/boids-simulating-birds-flock-behavior-in-python-9fff99375118.

[2] “Boids.” Online, 2025. Available: https://en.wikipedia.org/wiki/Boids, accessed: 14Nov2025.

[3] V. Adams, “Boids algorithm – augmented for distributed consensus.” Online. Available: https://vanhunteradams.com/Pico/Animal_Movement/Boids-algorithm.html.

[4] “Generative AI hulp voor visualisatie-code.” Online, 2025. Assistentie door generatieve AI tools voor het opstellen van visualisatie-code.